Six Missing Abstractions: Evidence from an Agent Workspace

Anonymous Author(s)

ABSTRACT

AI agents are becoming long-lived services that plan, invoke tools, and persist state, yet they are still hosted by application-layer workspaces built on operating-system abstractions designed for static workloads. We built and ran such a workspace—an open-source, multi-agent chat application with persistent per-agent memory—and report where its implementation hit walls that are missing OS abstractions. We enumerate six workarounds, measure two, and map each onto an OS abstraction that would remove it. The contribution is consumer-side evidence, not a new OS, for the set of abstractions the agent era is starved for.

1 INTRODUCTION

Operating systems expose processes, threads, files, sockets, and resource controllers as the units of execution, composition, and accounting. These abstractions were designed for workloads that are static, predictable, and short-lived relative to human attention. AI agents invert all three properties: they are long-lived, semantically adaptive, and interact with tools and environments in ways the runtime cannot predict. A recent workshop series argues that this mismatch is not incidental but structural, and that operating systems themselves must become *agentic* [1].

Most of the evidence offered for this claim comes from the *producer* side of the OS interface: new kernel primitives, new schedulers, new isolation models. This paper offers evidence from the *consumer* side. We built and operate a real, open-source agent workspace—a multi-agent chat application in which a human routes work to named agents via channel mentions, and each agent retains memory across sessions. The application is small (a few hundred lines) and states its non-goals explicitly. We found that at every hard problem, the limiting factor was not the application's logic but the absence of an operating-system abstraction that should plausibly exist.

This is a position paper, not a new system: the contribution is the mapping. Each of the workspace's admitted workarounds corresponds to an OS-level abstraction that would dissolve it. We measure two of these workarounds to show they are not cosmetic: a serial execution queue whose latency grows linearly with concurrency, and a prompt-replay memory scheme whose input-token cost grows linearly with history length. Each is technically re-implementable in user space, but only by re-building, per application, the scheduling and managed-state primitives that belong one layer down—and by getting their corner cases right every time.

2 THE WORKSPACE

The system is an open-source agent workspace (anonymized for review). A human types into a single chat channel; messages beginning with `@Name` are routed to the named agent. Each agent has a persona and a private, append-only memory. On each turn the agent's full history is replayed into the prompt, the model is invoked through a standard OpenAI-compatible endpoint, and the reply is stored back. Memory is isolated per agent: a fact told to one agent is not visible to another.

The implementation comprises a Node.js HTTP router, a SQLite store for channel history and per-agent memory, and a minimal web frontend. It uses the host Node runtime's built-in SQLite module; no external agent framework, vector database, or daemon is involved. The system is deployed for our own use.

What makes the system useful as evidence is not its capability but the explicitness of its non-goals. By design, it has:

- **No multi-agent concurrency.** Messages in a channel are processed through one serial queue. Two agents cannot act on the same state at the same time.
- **No memory compression.** The entire history is replayed into the prompt on every turn. There is no summarization or archival layer.
- **No inter-agent isolation.** All agents share one process and one database; there is no boundary preventing one agent from reading another's state.
- **No capability routing.** A mention is parsed by a regular expression; there is no matching of work to the agent best suited for it, and no delegation between agents.

These are not oversights. Each is a deliberate non-goal that kept the implementation readable. We now argue that each is also a place where an operating-system abstraction is conspicuously absent.

Table 1: Each workaround maps to an OS abstraction that would remove it.

Workaround (what is hand-rolled)	OS abstraction wanted
Serial queue (no concurrency)	Semantics-aware agent scheduling
Full-history replay (no compression)	Long-lived managed state
One process + one DB (no boundary)	Execute-only / attested isolation
JSON personas (hand-authored)	Skills as unit of composition
`@Name` regex (no matching)	Agent-facing interface / IPC
SQLite file (no managed object)	Agent state as OS object

3 WORKAROUNDS AND THE ABSTRACTIONS THEY WANT

Table 1 lists the six workarounds and the OS abstraction each implies. Two are measured below: single runs on a developer laptop against one OpenAI-compatible model (glm-5.1), reproducible from the artifact (anonymized for review).

2nd Workshop on Operating Systems Design for AI Agents (AgenticOS), September 29, 2026, Prague, Czechia
2026.

3.1 Measured: the serial queue

Because the workspace provides no agent scheduling, concurrent requests to one channel serialize. We fired N concurrent mentions at a channel for $N \in \{1, 2, 4, 8\}$ and measured end-to-end latency. Wall-clock time grew linearly: 3.3 s, 7.0 s, 13.9 s, 27.3 s for $N = 1, 2, 4, 8$—the eighth request waited over 27 seconds for seven predecessors. This is not a slow model; it is a *degenerate scheduler* that imposes a global order the application has no basis to choose. An OS that accounted for agents as first-class schedulable entities—with their intent, priority, and exploration semantics—could reorder, batch, or interleave these requests. The application can only fall back to a queue because no such abstraction exists beneath it.

3.2 Measured: prompt-replay memory

Because the workspace provides no managed memory abstraction, it replays the full conversation history into the prompt on every turn. We held an 8-turn conversation with one agent and read the model's reported input token count per turn. Prompt tokens grew 39, 81, 118, 142, 173, 206, 240, 308—tracking history length linearly, so each turn re-pays for every prior turn. An agent that persists across days or weeks, as the AgenticOS position assumes, becomes costly under this scheme well before it becomes unusable. The fix is not a better application prompt; it is a memory object the runtime manages: episodic storage with compression, recall, and eviction policies that the application requests rather than implements.

4 WHY THESE ARE ABSTRACTIONS, NOT FEATURES

A reviewer may object that the items in Table 1 are missing *features*, not missing *abstractions*. The distinction is the contribution, so we give a falsifiable test: *a requirement is an abstraction, not a feature, when any application-layer implementation of it is necessarily best-effort and cannot be verified correct in isolation.* A feature lives inside one program's logic and can be made correct there; an abstraction is a contract that must be guaranteed by the layer beneath, or not at all.

Isolation is the clearest case. An application can ask agents not to read each other's state, but any boundary it draws is advisory: a bug, a misconfigured tool call, or a prompt-injected agent can step over it, and the application has no vantage point from which to detect the crossing. Only the runtime, which mediates every memory access, can enforce a boundary that holds against a hostile or buggy agent. The same structure repeats across the rows: concurrency safety cannot be verified by a program that is itself one of the concurrent parties; durable, compressible context cannot be managed correctly by the process whose own state is being compressed; capability routing cannot be trusted when the router runs inside the routed process. In each case the application is both the enforcer and the enforced-upon, so its enforcement is best-effort by construction.

This is also why the walls are not specific to our workspace. Any application that lets agents persist, collaborate, or be selected by capability occupies the same position—enforcer and enforced-upon at once—and hits the same walls, because the walls are in the contract between the application and its runtime, not in the application's logic. The fact that a few-hundred-line application is forced to hand-roll all of these, badly, is the evidence.

5 RELATION TO THE AGENTICOS AGENDA

The mapping aligns with topics the AgenticOS community has already identified from the producer side. Agent-aware resource control [2] corresponds to our scheduling row; long-lived state abstractions for agent context and memory [1] correspond to our replay row; execute-only and attested-channel isolation [4, 5] corresponds to our isolation row; skills as a composition unit [3] corresponds to our persona row; agent-facing interface redesign [6] corresponds to our routing row. The agreement is the point: the abstractions an application is starved for are the same ones the OS community is beginning to build.

6 WHAT WE ARE NOT CLAIMING

We do not claim a new operating system. We do not claim that the abstractions named in Table 1 are the *correct* ones, only that they are *wanted*. We do not claim the workspace is representative of all agent applications; we claim that any application with its shape—persistent, multi-agent, capability-routed—will hit the same walls because the walls are in the runtime contract. We do not claim these abstractions are easy; we claim their absence is expensive, and we have the measurements to show it. The measurements are limited: single-run, one model, one agent, eight turns—no seeds, CIs, or hardware sweep. They show the growth *shape* of two walls, not their magnitude.

7 CONCLUSION

An application-layer agent workspace that states its non-goals explicitly is a measuring instrument for the abstractions its operating system lacks. We built one, measured two of its walls, and mapped six to OS-level abstractions the agent era needs. The hope is that consumer-side evidence of this kind complements the producer-side primitives the community is beginning to propose, and that the contract between agent applications and their runtime becomes something the application requests rather than re-implements.

REFERENCES

[1] AgenticOS Workshop Series. *Operating Systems Design for AI Agents.* SOSP 2026 (2nd edition); ASPLOS 2026 (1st edition). https://os-for-agent.github.io/

[2] Y. Zheng, J. Fan, Q. Fu, Y. Yang, W. Zhang, A. Quinn. *AgentCgroup: Understanding and Controlling OS Resources of AI Agents.* AgenticOS 2026.

[3] L. Chen, Z. Wang, W. Zheng, E. Feng, D. Du, Y. Xia, H. Chen. *Skills are the new Apps—Now It Is Time for Skill OS.* AgenticOS 2026.

[4] R. Tiwari, D. Williams. *Execute-Only Agents: Architectural Defense Against Prompt Injection for AI Agents.* AgenticOS 2026.

[5] Q. Wu, W. Zhang, G. Fang, et al. *Grimlock: Guarding High-Agency Systems with eBPF and Attested Channels.* AgenticOS 2026.

[6] Y. Wang, M. Li, H. Chen. *Rethinking OS Interfaces for LLM Agents.* AgenticOS 2026.